Optimization Problems

Many real-life problems are optimization problems *(not just decision problems)*

Typical techniques used in optimization problems:

- Dynamic programming (Week 10, early part of Week 11)
- Greedy Algorithm (Week 11)
- Approximation (Week 12)

Dynamic Programming

The basic idea

Fibonacci number

- Recursive formula:
  - F(1)=1
  - F(2)=1
  - F(n)=F(n-1)+F(n-2) for n≥3

Base case

Recursive case

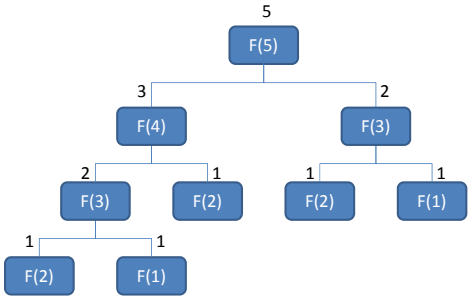
n	1	2	3	4	5	6	7	8	…
F(n)	1	1	2	3	5	8	13	21	…

Recursive Algorithm

Function F(n)

```
if n = 1 or n = 2 // base case
   q = 0;
else // recursive case
   q = F(n-1) + F(n-2);
return q;
```

Dry run of F(5)



F(3) is computed twice!

Analysis

- Note that T(n-1)≥T(n-2).
- Hence, $T(n) = T(n-1) + T(n-2) + c \geq 2T(n-2) + c$.

$$
\begin{aligned}
T(n) &\geq 2T(n-2) + c \\
&\geq 2^2 T(n - 2 * 2) + (1+2)c \\
&\geq 2^3 T(n - 3 * 2) + (1+2+2^2)c \\
&\ldots \\
&\ldots \\
&\geq 2^{n/2} T(0) + \left(1 + 2 + \cdots + 2^{n/2-1}\right)c \\
&= \Omega(2^{n/2})
\end{aligned}
$$

- The redundant computation leads to exponential time.

Tight bound

- $T(n) = T(n-1) + T(n-2) + c.$
- Solution: T(n) = $\Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) = \Omega(2^{n/2})$
  - Not covered in the course!

A better solution

```
Initialize a table F[1..n];
F[1]=1; F[2]=1;
for i = 3 to n
        F[i] = F[i-1] + F[i-2];
return F[n];
```

i	F[i]
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21

base case

- This solution runs in O(n) time.
- This technique is called dynamic programming.

CS3230 Lecture 10

Dynamic Programming

(ksung@comp.nus.edu.sg)

Lecture Topics and Readings (may spill to Week 11)

- CUT-ROD Problem, CLRS Ch15.1, "**Exponential" to Polynomial**
  - Steps in Dynamic Programming [CLRS] Ch15.1, CP3 Sec3.5
- 0-1 Knapsack Problem, **Pseudo-Polynomial**, CLRS Ex 16. 2-2, CP3 Sec3.5
- **Exponential** DP-TSP (review for ex CS2010), CP3 Sec3.5
- **Polynomial** *Bitonic* TSP, CLRS Problem 15-1, CP3 Sec9.3

How to avoid redundant computation in recursive algorithm?

Richard Bellman

Began the systematic study of dynamic programming (tabular method)

- The dynamic programming equation is also called Bellman equation

Also one of the inventor of Bellman-Ford's algorithm

- SSSP algorithm for a weighted directed graph
  - Runs in O(|V||E|) time in the worst case, usually can be terminated earlier in practice
  - Can be used to detect negative weight cycle in the graph

Rod Cutting Problem (CUT-ROD)

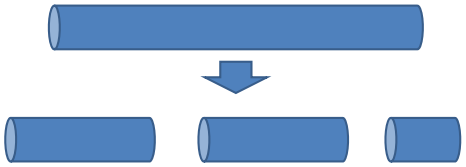
You have a rod of length **n**

You can cut it into pieces and sell the pieces

Assume that $\mathbf{p_i}$ ($\mathbf{p_i \geq 0}$) is the price for a rod of length **i**

You hope to maximize the profit

How should we cut the rod?



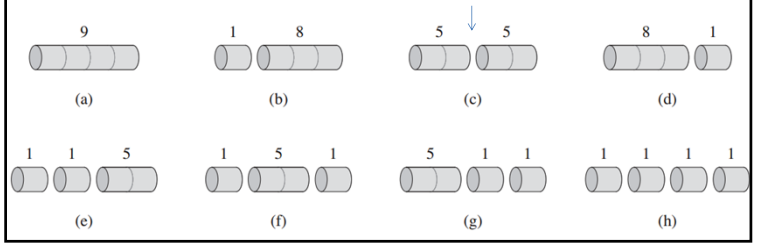
Example

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

INPUT: A rod of length 4

There are 8 possible ways to cut the rod as shown below

Optimal solution!



Problem Formulation of CUT-ROD

INPUT: **n** and $\mathbf{p_i}$ for **i** = 1, 2, …, n

OUTPUT: Among all possible ways to cut **n**
(i.e. all possible sets of lengths $\{i_1, i_2, \ldots, i_k\}$),
we aim to maximize $r_n = p_{i_1} + p_{i_2} + \cdots + p_{i_k}$

For the previous example, we have these observations:

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30
r_i	1	5	8	10	13	17	18	22	25	30
Pieces	1	2	3	2+2	2+3	6	1+6	2+6	3+6	10

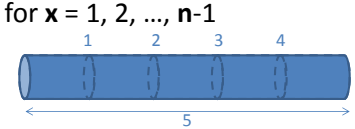
Simple (But Slow) Solution

INPUT: **n** and $\mathbf{p_i}$ for **i** = 1, 2, …, n

```
set r = p_n; // at least the price to sell with no cut
for each way to cut a length-n rod to {i_1, i_2, …, i_k}
   compute c = p_{i_1} + p_{i_2} + ··· + p_{i_k}; // O(n)
   if (c > r) // if this cut is more profitable
      r = c;
return r;
```

Why the Simple Solution is Slow?

For a length-**n** rod, we can choose <u>to cut</u> or <u>not to cut</u>
at position **x** for **x** = 1, 2, …, **n**-1



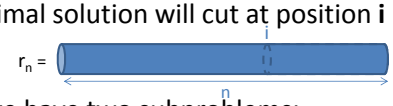
Hence, there are EXPONENTIAL: $\mathbf{2^{n-1}}$ ways to cut the rod
and the simple solution runs in $\mathbf{O(n * 2^{n-1})}$ time

- It is very slow…

Can we do better or is CUT-ROD ∈ NP-Complete?

Observation 1: Optimal Substructure

Consider a length-**n** rod, *suppose* we know that
the optimal solution will cut at position **i**



Then, we have two subproblems:

Observe that $r_n = r_i + r_{n-i}$

If overall optimal solution is formed by the optimal
solution of two (or more) related subproblems,
we say that the problem exhibits <u>optimal substructure</u>

Recursive Formula

However, we do NOT know the optimal cut!

No problem! We can check every cut!

Base case:

- $r_0 = 0$ (the revenue of having length-0 rod is obviously 0)

Recursive case:

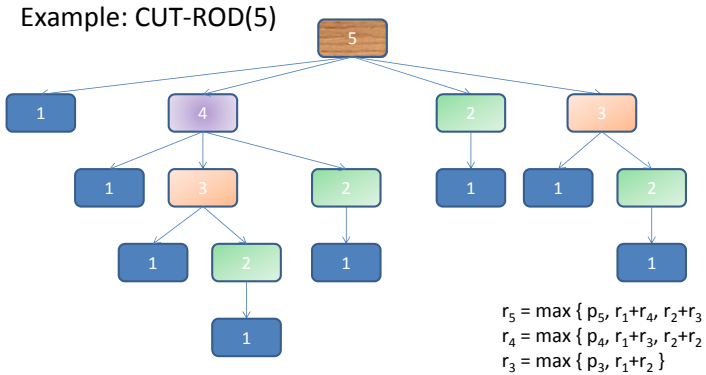
- $r_n = \max \{ p_n, r_1+r_{n-1}, r_2+r_{n-2}, \ldots, r_{n-2}+r_2, r_{n-1}+r_1 \}$
- $r_n = \max \{ p_n, r_1+r_{n-1}, r_2+r_{n-2}, \ldots, r_{\lfloor n/2 \rfloor}+r_{\lceil n/2 \rceil} \}$
- $r_n = \max \{ p_n, \max_{i=1,\ldots,\lfloor n/2 \rfloor} (r_i+r_{n-i}) \}$

Recursive Algorithm

```
CUT-ROD(n)
if n = 0 // base case
   q = 0;
else // recursive case
   q = pn; // price to sell with no cut
   for i = 1 to ⌊n/2⌋
     q = max{ q, CUT-ROD(i) + CUT-ROD(n-i) };
return q;
```

Observation 2: Overlapping Subproblems

Example: CUT-ROD(5)



$r_5 = \max \{ p_5, r_1+r_4, r_2+r_3 \}$
$r_4 = \max \{ p_4, r_1+r_3, r_2+r_2 \}$
$r_3 = \max \{ p_3, r_1+r_2 \}$
$r_2 = \max \{ p_2, r_1+r_1 \}$
$r_1 = \max \{ p_1 \}$

Notice the redundant **re-computations**

How to avoid redundant computation?

When there are overlapping computations, we can use dynamic programming instead of standard recursion

Idea: We store the computed results in a table

- For the second computation of the same CUT-ROD(n), we extract the stored result instead of calling recursion

This approach is called **memoized algorithm**

Memoized Algorithm

```
set r_i = -∞ for i = 0, 1, …, n; // initialize with -∞
output CUT-ROD-Memoized(n);

CUT-ROD-Memoized(n)
   if r_n ≥ 0 return r_n; // has been computed before
   if n = 0 // base case
      q = 0;
   else // recursive case
      q = p_n; // price to sell with no cut
      for i = 1 to ⌊n/2⌋
         q = max { q, CUT-ROD-Memoized(i) +
                      CUT-ROD-Memoized(n-i) };
   r_n = q;
   return q;
```

Bottom-up Dynamic Programming

CUT-ROD-DP(n)

```
r_0 = 0;
for j = 1 to n // start from small to large
   q = p_j;
   for k = 1 to ⌊j/2⌋
      q = max{ q, r_k + r_j-k };
   r_j = q;
return r_n;
```

Bottom-Up DP Demo

```
r_0 = 0; // base case
r_n = max{ p_n, max_i=1,…,⌊n/2⌋ (r_i+r_n-i) } // recursive case
```

More detailed example for r_5 =
$\max\{ p_5, \max_{i=1,2} (r_i+r_{5-i}) \}$ =
$\max \{10, \max(r_1+r_4, r_2+r_3)\}$ =
$\max \{10, \max(1+10, 5+8)\} = 13$

Length i	0	1	2	3	4	5	6	7	8	9	10
Price p_i		1	5	8	9	10	17	17	20	24	30
r_i	0	1	5	8	10	13	17	18	22	25	30

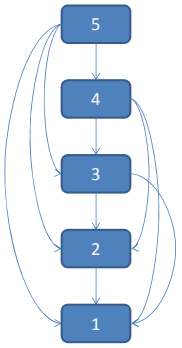
Subproblem (Directed Acyclic) Graph

A DP problem can be described as a subproblem (Directed Acyclic) Graph

- Each vertex is a subproblem
- For two subproblems x and y, (x, y) is an edge if x depends on y

$r_5 = \max \{ p_5, r_1+r_4, r_2+r_3 \}$
$r_4 = \max \{ p_4, r_1+r_3, r_2+r_2 \}$
$r_3 = \max \{ p_3, r_1+r_2 \}$
$r_2 = \max \{ p_2, r_1+r_1 \}$
$r_1 = \max \{ p_1 \}$

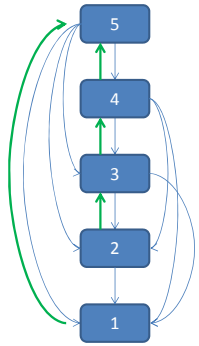
CUT-ROD(5)



Top-Down DP (Memoization)

Solve problem in DFS order

CUT-ROD(5)

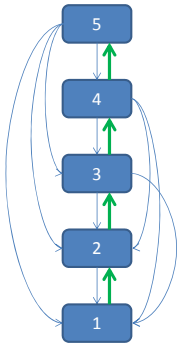


$r_5 = \max \{ p_5, \boxed{r_1}+\boxed{r_4}, \boxed{r_2}+\boxed{r_3} \}$
$r_4 = \max \{ p_4, \boxed{r_1}+\boxed{r_3}, \boxed{r_2}+r_2 \}$
$r_3 = \max \{ p_3, \boxed{r_1}+\boxed{r_2} \}$
$r_2 = \max \{ p_2, r_1+r_1 \}$
$r_1 = \max \{ p_1 \}$

Bottom-Up DP

Solve the problem in reverse topological order in the subproblem DAG

CUT-ROD(5)



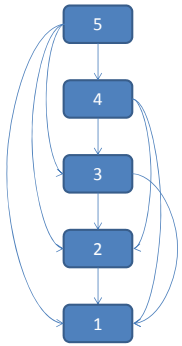
Running Time of CUT-ROD DP

Running time is the number of edges =
(n-1) + (n-2) + … + 1 =
$O(n^2)$
(both top-down and bottom-up DP)

Similar argument as in DFS/BFS analysis:

- We only touch each vertex and each edge once, and E > V in typical subproblem DAG

CUT-ROD(5)

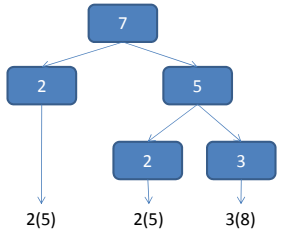


Reconstructing (One*) Optimal Solution

```
Call cut-rod-DP(n);
Call print-Cut-Rod(n);

print-Cut-Rod(n)
  if r_n = p_n
    print "i(p_n)";
  else
    for i = 1 to ⌊n/2⌋
      if (r_n = r_i+r_{n-i}) then break;
    print-Cut-Rod(i);
    print-Cut-Rod(n-i);
```

Example: n=7 (suppose $r_7=r_2+r_5$ although $r_7 = r_1+r_6$ too)



Length i	0	1	2	3	4	5	6	7	8	9	10
Price p_i		1	5	8	9	10	17	17	20	24	30
r_i	0	1	5	8	10	13	17	18	22	25	30

Recursion versus DP

All DP problem can be solved by just recursion

- i.e. if you can write recursive solution, you should already have the optimal subproblem property

So, when should we use DP?

- Answer: The recursive solution contains a *small number** of distinct subproblems <u>"repeated" (many time)</u>

* Later we will see that sometimes this is not necessarily polynomial…

Example (1)

$L(n) = L(n/2) + c_n$

Shall we use DP or recursion?

Recursion running time:

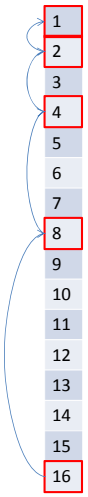
- O(log n)

(Bottom-Up) DP running time:

- O(n)

(Top-Down) DP running time:

- O(log n), but we waste O(n) of table space



Example (2)

$L(n) = L(\lfloor n/2 \rfloor)+\ldots+L(1) + c_n$

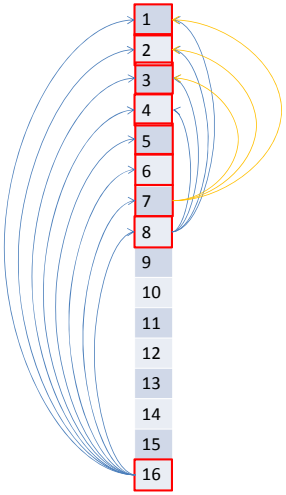
Shall we use DP or recursion?

Recursion running time:

- Super-polynomial time

DP (either version) running time:

- $O(n^2)$



Current Summary for DP Technique

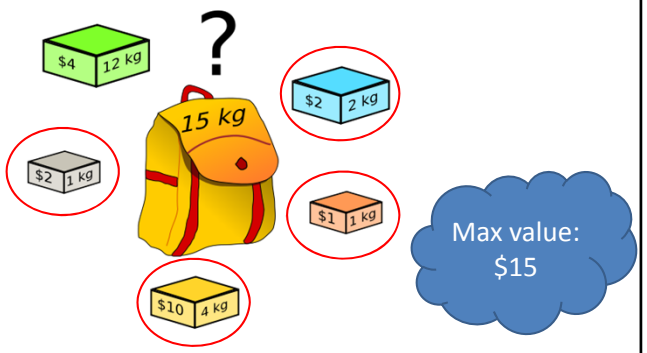
Steps:

1. Characterize the structure of an optimal solution
2. Define the recursive formula and check if there are small number of distinct subproblems repeated many times
3. Develop the algorithm to fill-in the dynamic programming table (either with top-down or bottom-up strategy)
4. Reconstruct the optimal solution (optional)

0-1 Knapsack Problem

Introducing the term
Pseudo-Polynomial

0-1 Knapsack Problem



What is the maximum value you can get?

0-1 Knapsack (CLRS Ex 16.2-2)

This problem finds application in resource allocation

Formal definition:

INPUT: $(v_1, w_1), (v_2, w_2), \ldots, (v_n, w_n)$, and W

OUTPUT: A subset $S \subseteq \{1, 2, \ldots, n\}$ that maximizes $\sum_{i \in S} v_i$ such that $\sum_{i \in S} w_i \leq W$

Simple (but Slow) Solution

Knapsack($\{v_1, v_2, \ldots, v_n\}$, $\{w_1, w_2, \ldots, w_n\}$, W)

```
maxV = 0;
for every subset S ⊆ {1, 2, …, n},
    if Σ_{i∈S} w_i ≤ W and Σ_{i∈S} v_i > maxV, then
        maxV = Σ_{i∈S} v_i;
return maxV;
```

Running time:

- There are 2^n possible subsets S of $\{1, 2, \ldots, n\}$
- It takes O(n) time to compute $\sum_{i \in S} w_i$ and $\sum_{i \in S} v_i$
- Hence, the running time is $O(n * 2^n)$

Knapsack Problem is in NPC

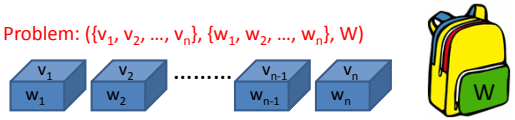
(0-1) Knapsack problem is NP-Complete

- Proof omitted (it is related to SUBSET-SUM)
- (see your HW3 later)

When the weights $\mathbf{w_i}$ are integers and **W** is bounded, can we solve the problem in polynomial time?

Step 1: Optimal substructure

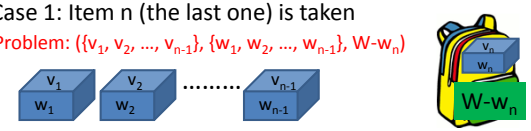
Problem: $(\{v_1, v_2, \ldots, v_n\}, \{w_1, w_2, \ldots, w_n\}, W)$



Two cases:

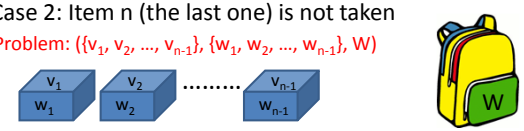
- Case 1: Item n (the last one) is taken

  Problem: $(\{v_1, v_2, \ldots, v_{n-1}\}, \{w_1, w_2, \ldots, w_{n-1}\}, W\text{-}w_n)$



- Case 2: Item n (the last one) is not taken

  Problem: $(\{v_1, v_2, \ldots, v_{n-1}\}, \{w_1, w_2, \ldots, w_{n-1}\}, W)$



Observation

Good news so far!

- For both cases, the problems are <u>smaller</u> knapsack problems

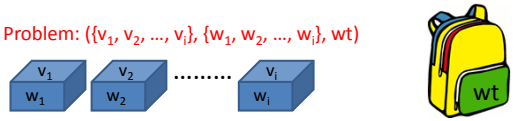
Hence, knapsack problem exhibits optimal substructure

- This is one of the key requirement for DP

Recursive formula for Knapsack (1)

Let **m(i, wt)** be the max value that can be attained using subset of items in **{1, 2, …, i}** whose total weight $\leq$ **wt**

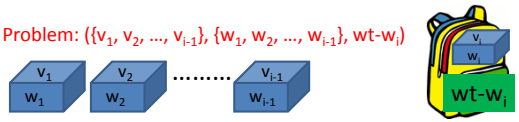
Problem: $(\{v_1, v_2, \ldots, v_i\}, \{w_1, w_2, \ldots, w_i\}, wt)$



Recursive formula for Knapsack (2)

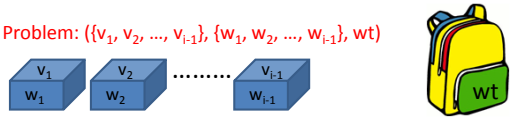
Recursive case 1: **$m(i, wt) = m(i-1, wt-w_i) + v_i$**
if **$w_i \leq wt$**

Problem: $(\{v_1, v_2, \ldots, v_{i-1}\}, \{w_1, w_2, \ldots, w_{i-1}\}, wt-w_i)$



Recursive case 2: **$m(i, wt) = m(i-1, wt)$**

Problem: $(\{v_1, v_2, \ldots, v_{i-1}\}, \{w_1, w_2, \ldots, w_{i-1}\}, wt)$

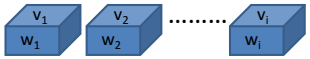


Recursive formula for Knapsack (3)

Base case 1: If **$i = 0$**, then **$m(i, wt) = 0$**

No item!

Base case 2: If **$wt = 0$**, then **$m(i, wt) = 0$**



Recursive formula for Knapsack (4)

Recursive cases:

$$m(i, wt) = max \begin{cases} m(i-1, wt-w_i) + v_i \; if \; w_i \leq wt \\ m(i-1, wt) \end{cases}$$

Base cases:

If i = 0, then m(i, wt) = 0

If wt = 0, then m(i, wt) = 0

Recursive algorithm

```
KS_recur(i, wt)
     if (i = 0)  val = 0; // base case 1
else if (wt = 0) val = 0; // base case 2
else
  val = KS_recur(i-1, wt); // rec case 2: not take
  if (wt > wi), then // rec case 1: take
    val = max{ val, KS_recur(i-1, wt-wi)+vi };
  return val;
```

INPUT: $\{v_1, v_2, \ldots, v_n\}$, $\{w_1, w_2, \ldots, w_n\}$ and W

```
return KS_recur(n, W);
```

Time analysis

Time(n, W) = Time(n-1, W) + Time(n-1, W-w_n) + O(1)

Assume W ≥ w_1+w_2+…+w_n, i.e. the knapsack is big

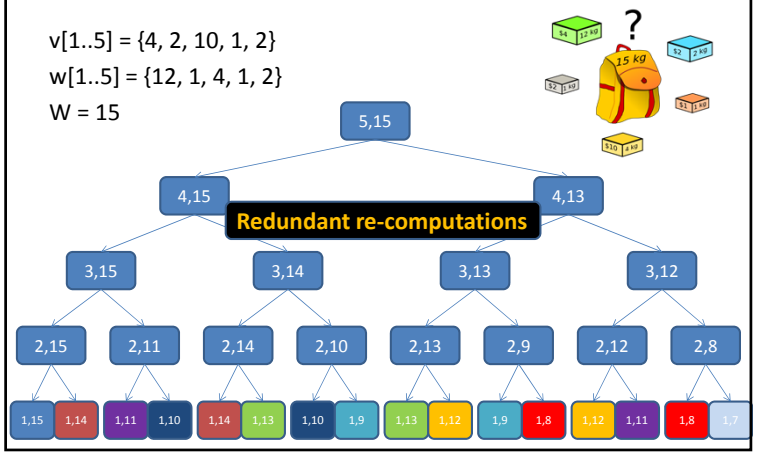
Time(n, W) = O(2^n)… erm…

Why the algorithm is slow?

v[1..5] = {4, 2, 10, 1, 2}

w[1..5] = {12, 1, 4, 1, 2}

W = 15



Memoized (Top-down DP) solution

```
KS_memoized(i, wt)
if (m[i, wt] ≥ 0) return m[i, wt];
     if (i = 0)  val = 0; // base case 1
else if (wt = 0) val = 0; // base case 2
else
  val = KS_memoized(i-1, wt); // rec case 2: not take
  if (wt > w_i), then // rec case 1: take
    val = max{ val, KS_memoized(i-1, wt-w_i)+v_i };
m[i, wt] = val;
return val;
```

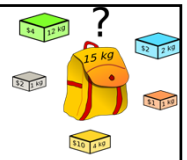
INPUT: {v_1, v_2, …, v_n}, {w_1, w_2, …, w_n} and W

```
set m[i, w] = -∞
for i=1,2,…,n and w = 0,1,…,W;
return KS_memoized(n, W);
```

Bottom-up DP solution

```
DP_KS({v_1, v_2, …, v_n}, {w_1, w_2, …, w_n}, W)
for w = 0 to W // step 1 (base case 1)
  m[0, w] = 0;
for i = 1 to n // step 2 (base case 2)
  m[i, 0] = 0;
for i = 1 to n // step 3 (the most complex one)
  for j = 0 to W
    if j >= w_i then // rec case 1: take
      m[i, j] = max(m[i-1, j], m[i-1, j-w[i]] + vi);
    else // rec case 2: not take
      m[i, j] = m[i-1, j];
return m[n, W]; // step 4
```

Demo (1)

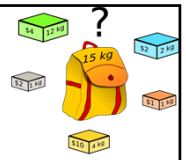


v[1..5] = {4, 2, 10, 1, 2}
w[1..5] = {12, 1, 4, 1, 2}
W = 15

```
for w = 0 to W
   m[0, w] = 0;
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1																
2																
3																
4																
5																

Demo (2)

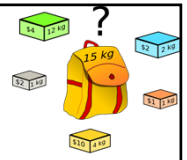


v[1..5] = {4, 2, 10, 1, 2}
w[1..5] = {12, 1, 4, 1, 2}
W = 15

```
for i = 1 to n
   m[i, 0] = 0;
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0															
2	0															
3	0															
4	0															
5	0															

Demo (3)



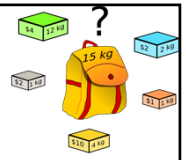
v[1..5] = {4, 2, 10, 1, 2}
w[1..5] = {12, 1, 4, 1, 2}
W = 15

```
for i = 1 to n
  for j = 0 to W
    if j >= wi then
      m[i, j] = max(m[i-1, j],
          m[i-1, j-w[i]] + vi);
    else
      m[i, j] = m[i-1, j];
```

E.g. m[3,5] = max(m[3-1,5], m[3-1,5-w_3]+v_3)
= max(m[2,5],m[2,1]+10)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	12										
4																
5																

Demo (4)



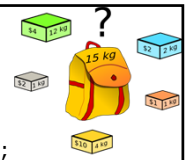
v[1..5] = {4, 2, 10, 1, 2}
w[1..5] = {12, 1, 4, 1, 2}
W = 15

```
for i = 1 to n
  for j = 0 to W
    if j >= wi then
      m[i, j] = max(m[i-1, j],
          m[i-1, j-w[i]] + vi);
    else
      m[i, j] = m[i-1, j];
```

E.g. m[4,5] = max(m[4-1,5], m[4-1,5-w_4]+v_2)
= max(m[3,5], m[3,4]+1)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
4	0	2	3	3	10	12										
5																

Demo (5)



v[1..5] = {4, 2, 10, 1, 2}

w[1..5] = {12, 1, 4, 1, 2}

W = 15

```
return m[n, W];
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
4	0	2	3	3	10	12	13	13	13	13	13	13	13	13	13	13
5	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Running time

Step 1: O(W) time

Step 2: O(n) time

Step 3: O(nW) time

Step 4: O(1) time

In total: O(nW) time

Reconstruct the set of items

- Call printSet(n,W);
- printSet(i,wt)
  - If m(i,wt) = m(i-1,wt-w_i)+v_i, then
    - return printSet(i-1, wt-w_i)∪{i};
  - Else // m(i,wt)=m(i-1,wt)
    - return printSet(i-1,wt);

- Example: W=15
  - v[1..5] = {4, 2, 10, 1, 2}
  - w[1..5] = {12, 1, 4, 1, 2}
- m(5,15)=m(4,15-2)+2
- m(4,13)=m(3,13-1)+1
- m(3,12)=m(2,12-4)+10
- m(2,8)=m(1,8-1)+2
- m(1,7)=m(0,7)

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
✓	2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
✓	3	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
✓	4	0	2	3	3	10	12	13	13	13	13	13	13	13	13	13	13
✓	5	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Is Knapsack problem in P?

We have showed that Knapsack problem can be solved in O(nW) time, is it a polynomial algorithm?

- Only when W is O(n), then it runs in polynomial time

This DP solution is usually referred as a **pseudo-polynomial** time algorithm

- Actually, Knapsack problem is not in P since W can have O(n) digits, i.e. W = $10^{O(n)}$
- In such case, this algorithm runs in exponential time

Traveling Salesman Problem

A DP algorithm can still have exponential time complexity…

Traveling Salesman Problem (TSP)

dist	0	1	…	n-1
0				
1				
…				
n-1				

The decision version of this problem is shown to be in NP-Complete (Lecture 08)

Indeed, there are O(N!) possible tours to try…

Brute-force TSP

- To find optimal solution,
  - we need to try all n! tours and find the best one.
- This solution requires $\Omega(n!) = n^{\Omega(n)}$ time.
- Can we do better?

Are all n! permutations distinct?

Yes, but the <u>partial tours</u> can overlap:

Example, for an instance with N cities:

1. Tour 1: A-B-C-**<u>(N-3) other cities</u>**-return to A
2. Tour 2: A-C-B-**<u>the same (N-3) other cities</u>**-return to A

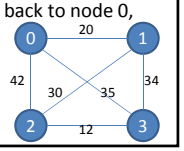
Overlapping subproblems!

Bitmask technique

- Let {0, 1, 2, …, n-1} be the set of n nodes.
- For any two nodes p and q, let $d(p,q)$ be the distance between nodes p and q.
- Let $b_1 b_2 \ldots b_{n-1}$ be a length-(n-1) bit vector such that $b_p = 1$ indicates node p is visited.

Recursive formula

- Define $tsp(p, b_1 b_2 \ldots b_{n-1})$ be the minimum cost tour that start from a node p, visit all the nodes q with $b_q = 0$ and finally visit node 0.
- Our aim is to compute tsp(0, 0…0).
- Examples:
  - $tsp(0{,}000)$ means the shortest distance from node 0, visit nodes 1, 2 and 3 and finally go back to node 0.
  - $tsp(1{,}100)$ means the shortest distance from node 1, visit nodes 2 and 3 and finally go back to node 0.
  - $tsp(3{,}101)$ means the shortest distance from node 3, visit node 2 and finally go back to node 0, which is d(3, 2)+d(2,0).
  - tsp(2,111) means the shortest distance from node 2 go back to node 0, which is d(2, 0).



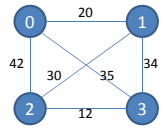
Recursive equation

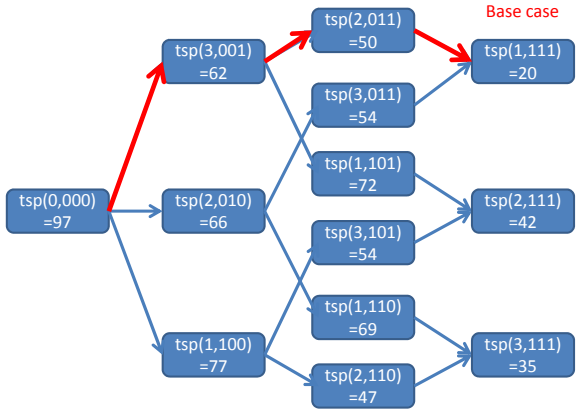
- Base case ($b_1 b_2 \ldots b_{n-1} = 1 \ldots 1$):
  - $tsp(p, 11 \ldots 1) = d(p, 0)$ for any node p
- Recursive case ($b_1 b_2 \ldots b_{n-1} \neq 1 \ldots 1$ & b_p=1):
  - $tsp(p, b_1 b_2 \ldots b_{n-1}) = \min_{q=1,2,\ldots,n-1 \wedge q \neq p \wedge b_q = 0} \{ tsp(q, b_1 \ldots b_{q-1} 1 b_{q+1} \ldots b_{n-1}) + d(p,q) \}$

DP algorithm

- for p = 1 to n-1
  - $tsp(p, 11 \ldots 1) = d(p, 0)$
- for z= n-1 to 1,
  - for all bit patterns $b_1 b_2 \ldots b_{n-1}$ with z's ones,
    - for every p such that $b_p = 1$,
      - $tsp(p, b_1 b_2 \ldots b_{n-1}) = \min_{q=1,2,\ldots,n-1 \wedge q \neq p \wedge b_q = 0} \{ d(p,q) + tsp(q, b_1 \ldots b_{q-1} 1 b_{q+1} \ldots b_{n-1}) \}$
- $tsp(0{,}0 \ldots 0) = \min_{0 \le q \le n-1} \{ d(0,q) + tsp(q, 0^{q-1} 1 0^{n-q}) \}$

Bottom-up DP





Analysis of Top-Down TSP DP

How many vertices are there in this subproblems DAG?

- Up to $O(N * 2^N)$
  - N is the current vertex
  - 2^N is the size of the set of visited vertices

How many edges are there in this subproblems DAG?

- Up to O(N) outgoing edges per vertex as we must check all neighbors of a vertex as the input is a complete graph, each vertex has out-degree N-1
- $O((N * 2^N) * N) = O(N^2 * 2^N)$, still exponential :O
  - But still faster than O(N!)

More DP problems in [CLRS/CP3]

On **P** problems

- Range Sum
- Matrix-chain multiplication
- Longest common subsequence
- Optimal binary search tree
- All-Pairs Shortest Path (Floyd Warshall's)
- (Small) Graph Matching* (DP solution turns this P problem to exponential)

On **NP** problems (pseudo-polynomial/exponential/special-case)

- SUBSET-SUM (similar with Knapsack)
- COIN-CHANGE
- INDEPENDENT-SET/VERTEX-COVER (on tree)

Summary

- Dynamic Programming is a powerful problem solving technique for problems that exhibit:
  1. Optimal Substructure
  2. Overlapping Subproblems
- We can use Top-Down or Bottom-Up DP formulation
  - Simplified time complexity analysis:
    - Bottom-up: # of cells * effort to fill that cell
    - Top-down: # of vertices * # out-edges/vertex in recursion DAG
- DP solution can still have pseudo-polynomial or exponential time complexity!